

DM—Assignment 1

Name：張東浩

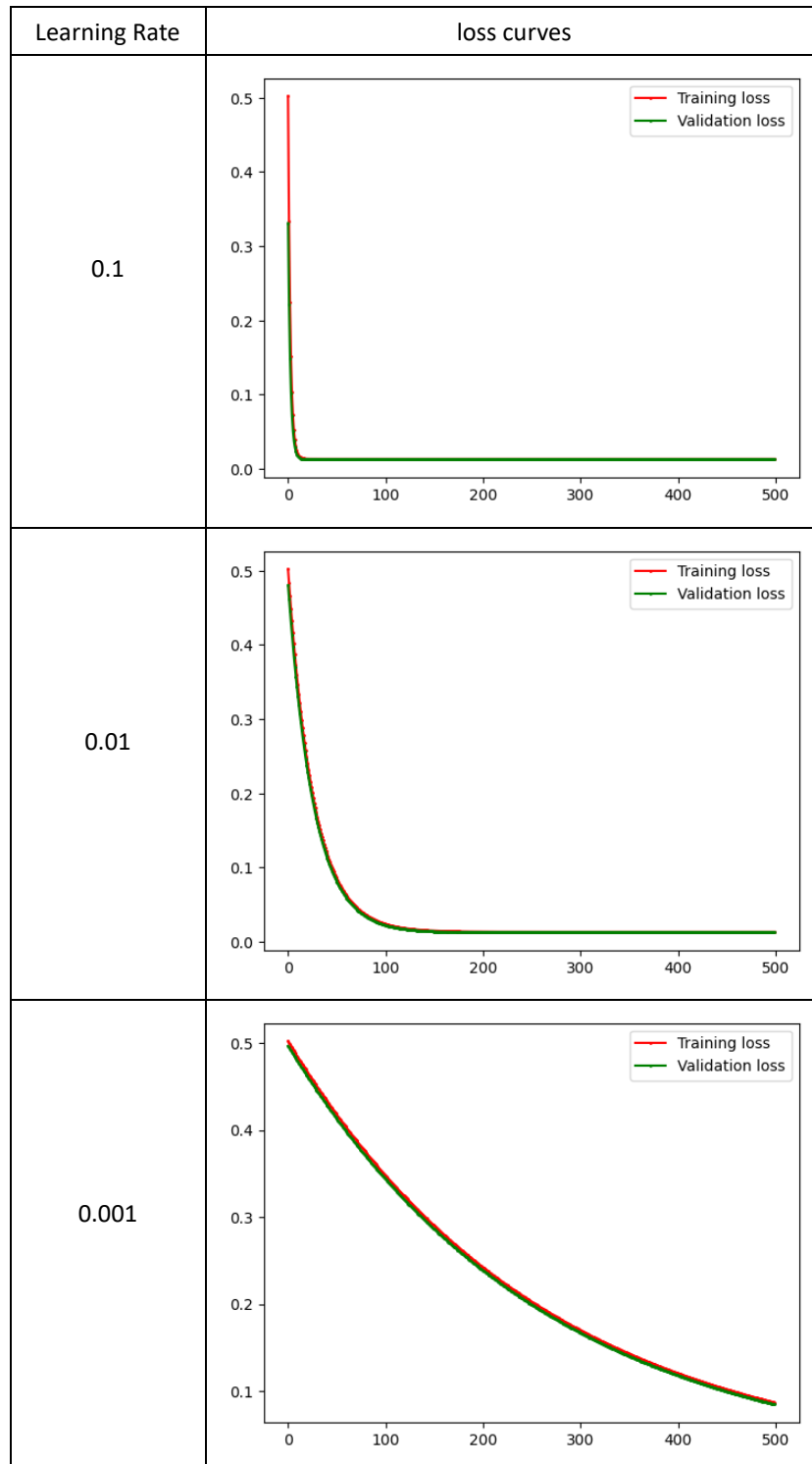
Student ID：431551038

GitHub：<https://github.com/doctor-66/535703-Data-Mining-Assignment1>

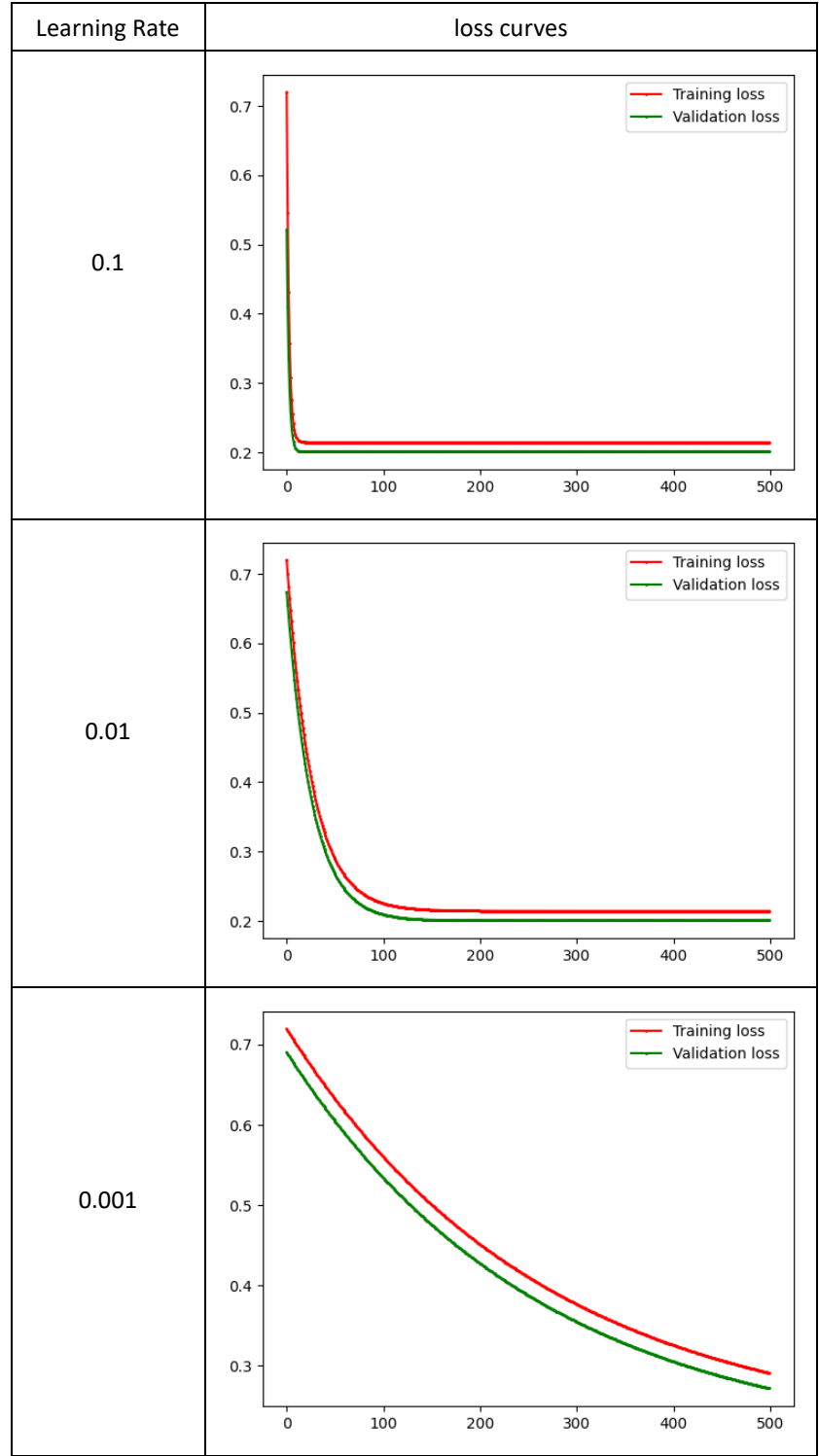
1. Linear Regression：

1. loss curve in different learning rate

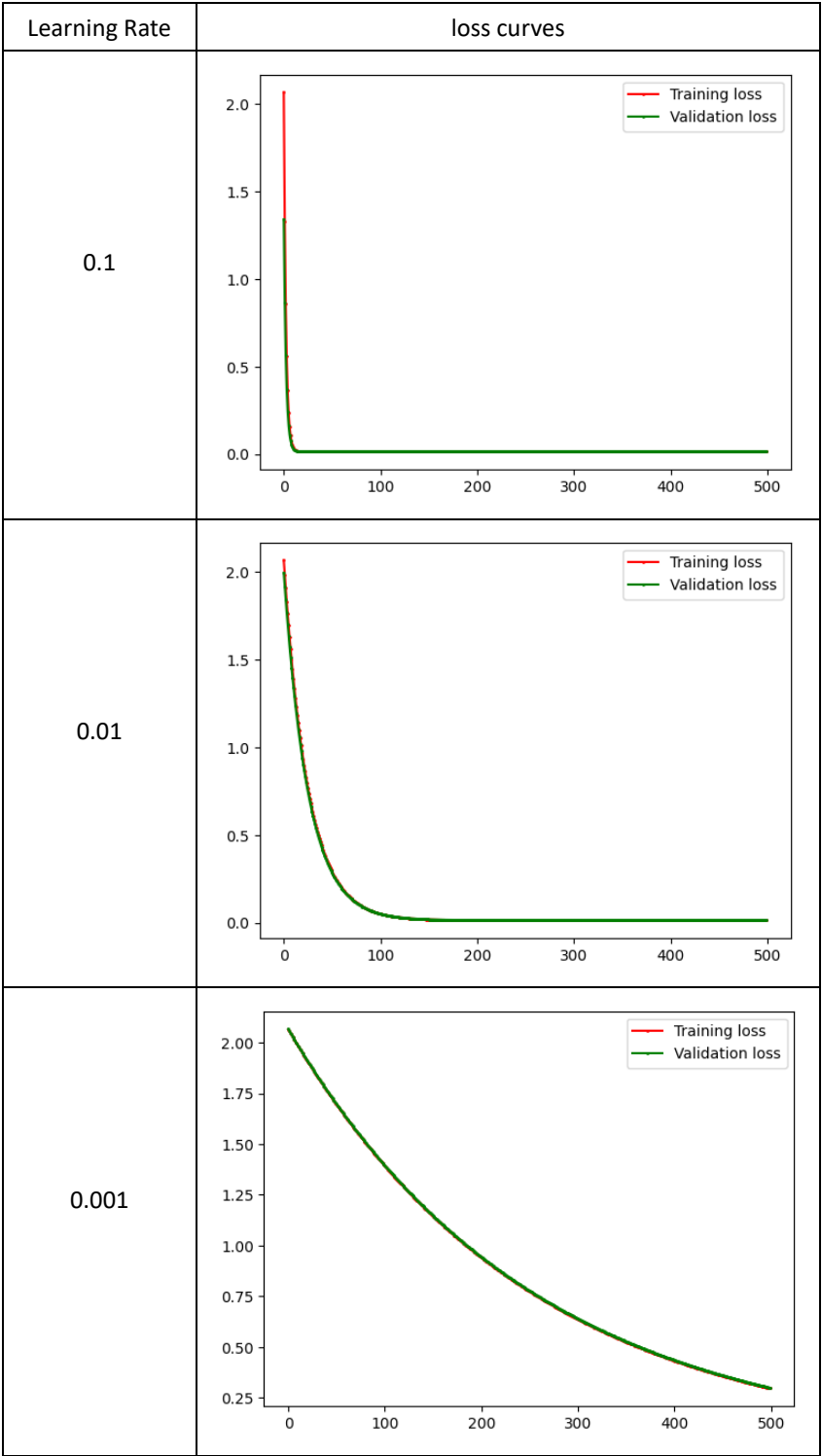
Datase："linear_data_A"



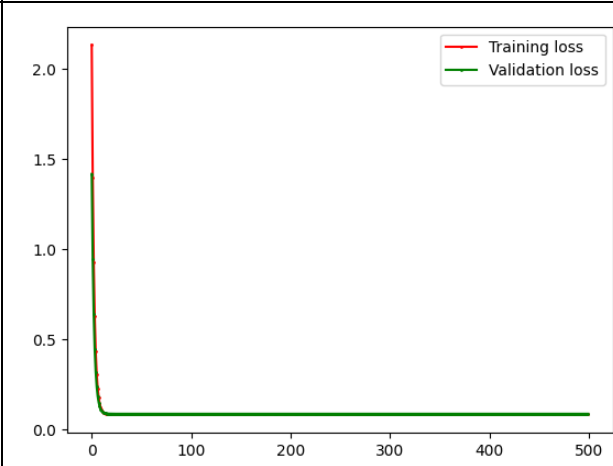
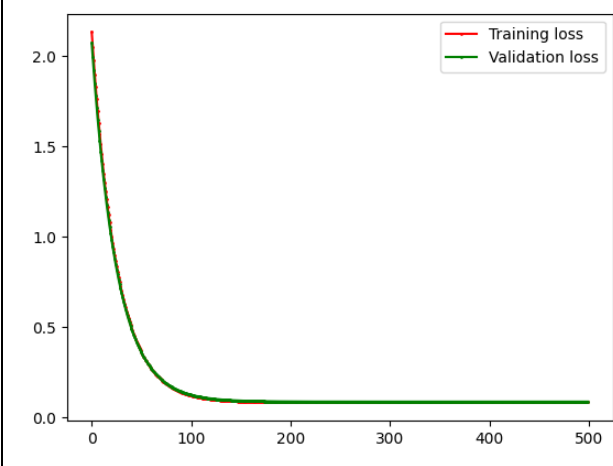
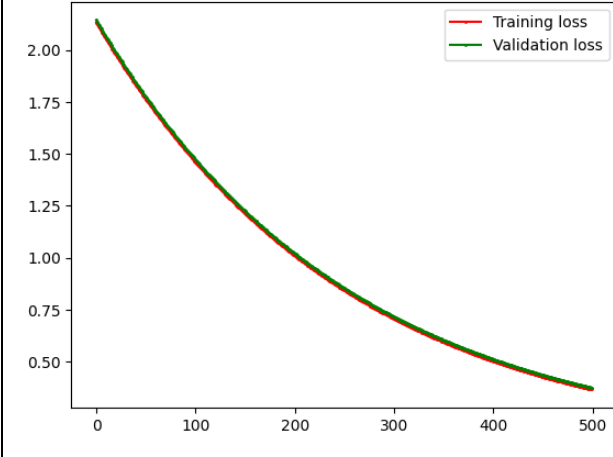
Dataste : “linear_data_B”



Dataste : "linear_data_C"



Dataste : “linear_data_D”

Learning Rate	loss curves																								
0.1	 The plot shows training loss (red line) and validation loss (green line) over 500 iterations for a learning rate of 0.1. Both losses start at approximately 2.1 and drop sharply to near zero by iteration 10, remaining stable thereafter. <table><tr><th>Iteration</th><th>Training loss</th><th>Validation loss</th></tr><tr><td>0</td><td>2.1</td><td>2.1</td></tr><tr><td>10</td><td>0.1</td><td>0.1</td></tr><tr><td>100</td><td>0.1</td><td>0.1</td></tr><tr><td>200</td><td>0.1</td><td>0.1</td></tr><tr><td>300</td><td>0.1</td><td>0.1</td></tr><tr><td>400</td><td>0.1</td><td>0.1</td></tr><tr><td>500</td><td>0.1</td><td>0.1</td></tr></table>	Iteration	Training loss	Validation loss	0	2.1	2.1	10	0.1	0.1	100	0.1	0.1	200	0.1	0.1	300	0.1	0.1	400	0.1	0.1	500	0.1	0.1
Iteration	Training loss	Validation loss																							
0	2.1	2.1																							
10	0.1	0.1																							
100	0.1	0.1																							
200	0.1	0.1																							
300	0.1	0.1																							
400	0.1	0.1																							
500	0.1	0.1																							
0.01	 The plot shows training loss (red line) and validation loss (green line) over 500 iterations for a learning rate of 0.01. Both losses start at approximately 2.1 and decrease more gradually than in the 0.1 case, reaching near zero by iteration 100. <table><tr><th>Iteration</th><th>Training loss</th><th>Validation loss</th></tr><tr><td>0</td><td>2.1</td><td>2.1</td></tr><tr><td>100</td><td>0.1</td><td>0.1</td></tr><tr><td>200</td><td>0.1</td><td>0.1</td></tr><tr><td>300</td><td>0.1</td><td>0.1</td></tr><tr><td>400</td><td>0.1</td><td>0.1</td></tr><tr><td>500</td><td>0.1</td><td>0.1</td></tr></table>	Iteration	Training loss	Validation loss	0	2.1	2.1	100	0.1	0.1	200	0.1	0.1	300	0.1	0.1	400	0.1	0.1	500	0.1	0.1			
Iteration	Training loss	Validation loss																							
0	2.1	2.1																							
100	0.1	0.1																							
200	0.1	0.1																							
300	0.1	0.1																							
400	0.1	0.1																							
500	0.1	0.1																							
0.001	 The plot shows training loss (red line) and validation loss (green line) over 500 iterations for a learning rate of 0.001. Both losses start at approximately 2.1 and decrease very slowly, reaching approximately 0.4 by iteration 500. <table><tr><th>Iteration</th><th>Training loss</th><th>Validation loss</th></tr><tr><td>0</td><td>2.1</td><td>2.1</td></tr><tr><td>100</td><td>1.5</td><td>1.5</td></tr><tr><td>200</td><td>1.0</td><td>1.0</td></tr><tr><td>300</td><td>0.7</td><td>0.7</td></tr><tr><td>400</td><td>0.5</td><td>0.5</td></tr><tr><td>500</td><td>0.4</td><td>0.4</td></tr></table>	Iteration	Training loss	Validation loss	0	2.1	2.1	100	1.5	1.5	200	1.0	1.0	300	0.7	0.7	400	0.5	0.5	500	0.4	0.4			
Iteration	Training loss	Validation loss																							
0	2.1	2.1																							
100	1.5	1.5																							
200	1.0	1.0																							
300	0.7	0.7																							
400	0.5	0.5																							
500	0.4	0.4																							

2. evaluation results in different learning rate

Datase : "linear_data_A"

Learning rate	Results
0.1	<pre> === Linear Regression Evaluation === MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690 </pre>
0.01	<pre> === Linear Regression Evaluation === MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690 </pre>
0.001	<pre> === Linear Regression Evaluation === MSE : 0.0832 MAE : 0.2369 RMSE : 0.2884 R-squared : -1.6836 </pre>

Datase : "linear_data_B"

Learning rate	Results
0.1	<pre> === Linear Regression Evaluation === MSE : 0.2137 MAE : 0.4038 RMSE : 0.4623 R-squared : 0.0448 </pre>
0.01	<pre> === Linear Regression Evaluation === MSE : 0.2137 MAE : 0.4038 RMSE : 0.4623 R-squared : 0.0448 </pre>
0.001	<pre> === Linear Regression Evaluation === MSE : 0.2687 MAE : 0.4404 RMSE : 0.5184 R-squared : -0.2011 </pre>

Datase : "linear_data_C"

Learning rate	Results
0.1	<pre> === Linear Regression Evaluation === MSE : 0.0132 MAE : 0.0987 RMSE : 0.1148 R-squared : 0.9954 </pre>

0.01	<pre> === Linear Regression Evaluation === MSE : 0.0132 MAE : 0.0987 RMSE : 0.1148 R-squared : 0.9954 </pre>
0.001	<pre> === Linear Regression Evaluation === MSE : 0.2852 MAE : 0.4273 RMSE : 0.5341 R-squared : 0.8994 </pre>

Datase : "linear_data_D"

Learning rate	Results
0.1	<pre> === Linear Regression Evaluation === MSE : 0.0824 MAE : 0.2467 RMSE : 0.2870 R-squared : 0.9716 </pre>
0.01	<pre> === Linear Regression Evaluation === MSE : 0.0824 MAE : 0.2467 RMSE : 0.2870 R-squared : 0.9716 </pre>
0.001	<pre> === Linear Regression Evaluation === MSE : 0.3540 MAE : 0.4778 RMSE : 0.5950 R-squared : 0.8780 </pre>

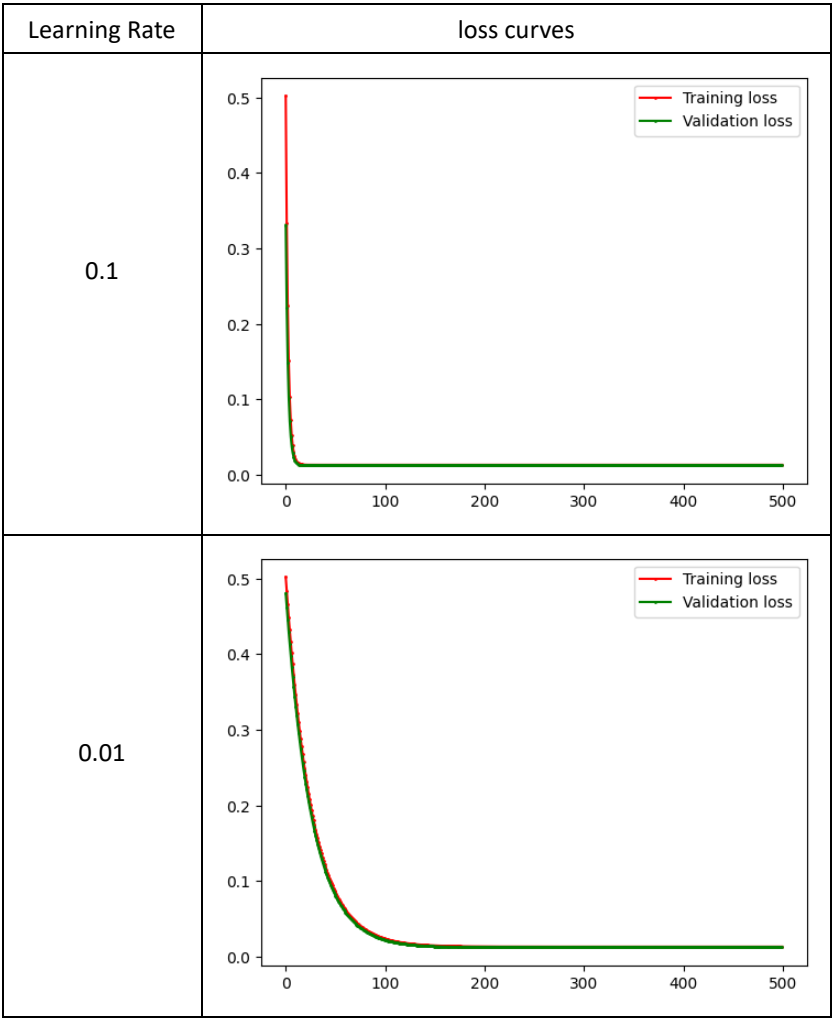
1. Observations

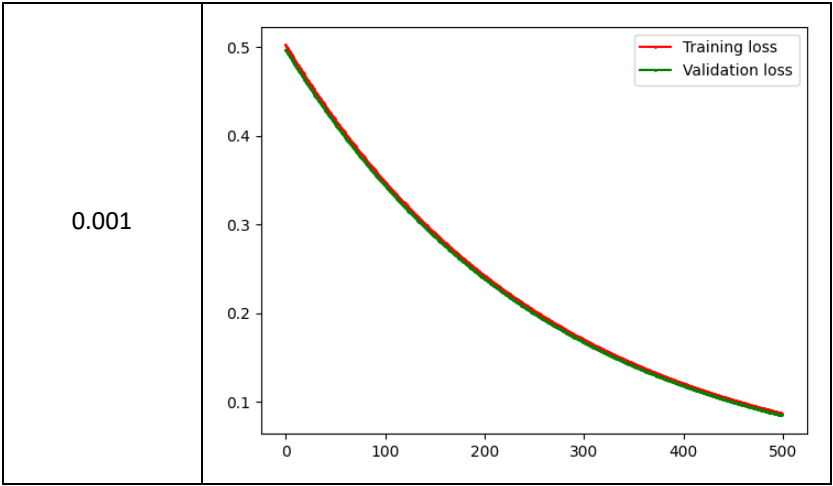
In my observations, Across the four datasets, the results show a consistent relationship between the loss curves and the final evaluation metrics. Overall, learning rates of **0.1** and **0.01** both converge well and achieve clearly better regression performance than **0.001**, while **0.1** is the most efficient choice because it reaches the same or comparable final results with much faster convergence. Among the four datasets, "**linear_data_C**" achieves the best overall performance, with the lowest error values (**MSE = 0.0132**, **MAE = 0.0987**, **RMSE = 0.1148**) and the highest **R-squared = 0.9954** under both **0.1** and **0.01**. This strong evaluation result is fully consistent with its loss curves, where both training and validation losses drop rapidly and stabilize near zero. In comparison, "**linear_data_D**" also performs well with a high **R-squared = 0.9716**, but its errors remain higher than those of "**linear_data_C**", while "**linear_data_A**" shows moderate performance and "**linear_data_B**" is the weakest among the four, with much larger errors and a very low **R-squared = 0.0448**

even under its best learning-rate settings. For all datasets, **learning rate = 0.001** leads to much slower loss reduction and worse evaluation scores, indicating insufficient convergence within the fixed training budget. Therefore, considering both effectiveness and efficiency, the best overall combination is **“linear_data_C”** with **learning rate = 0.1**, since it delivers the strongest predictive performance while also converging the fastest.

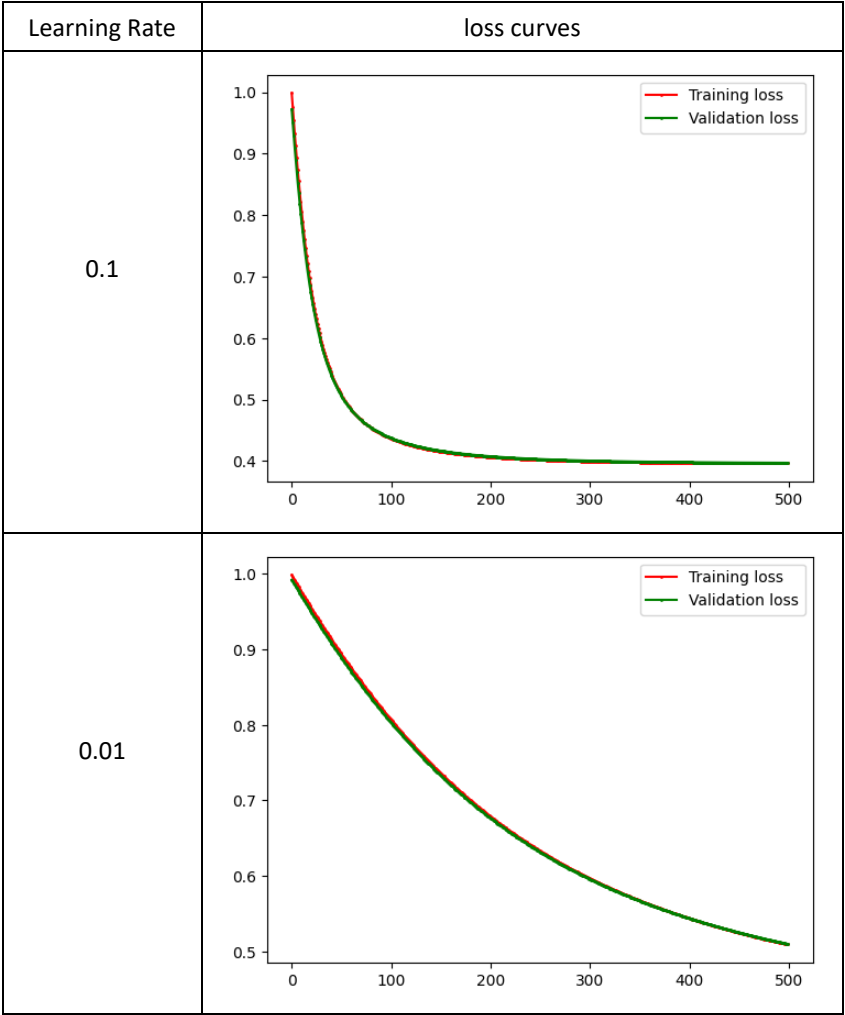
2. Logistic Regression :

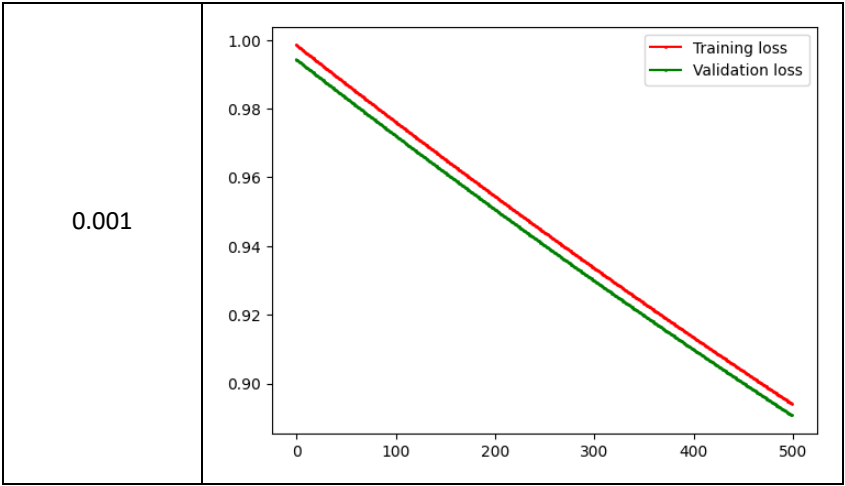
Dataste : “logistic_data_A”



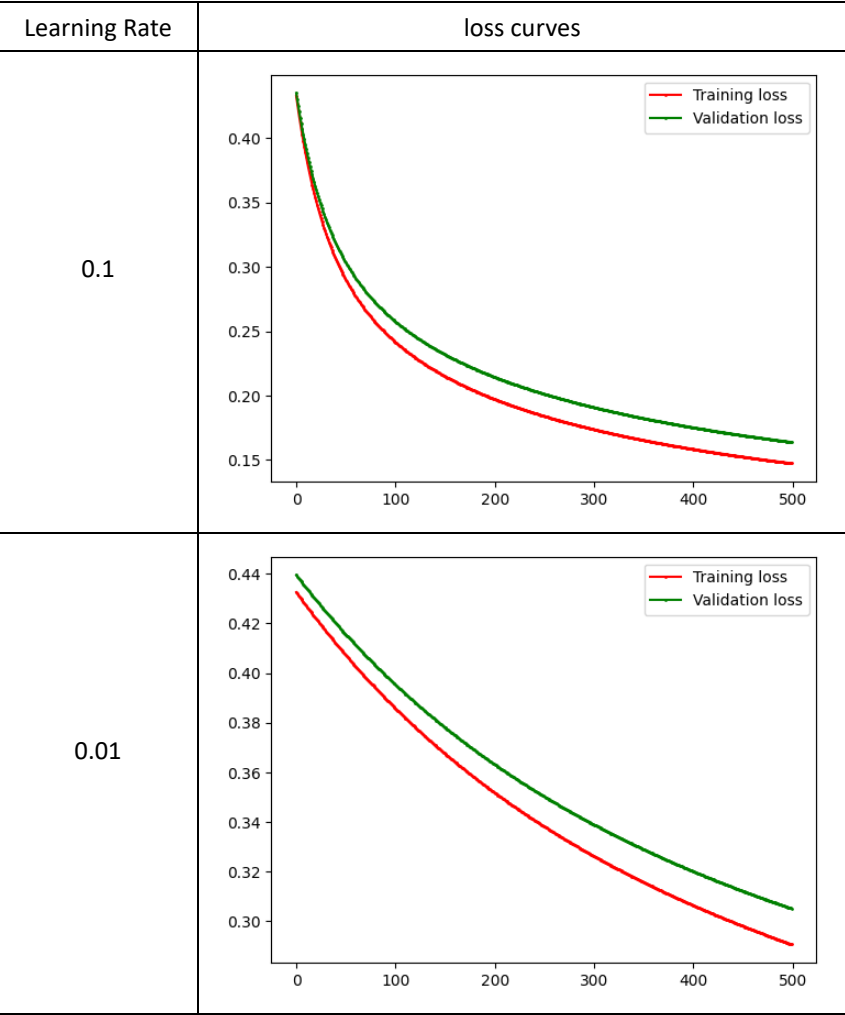


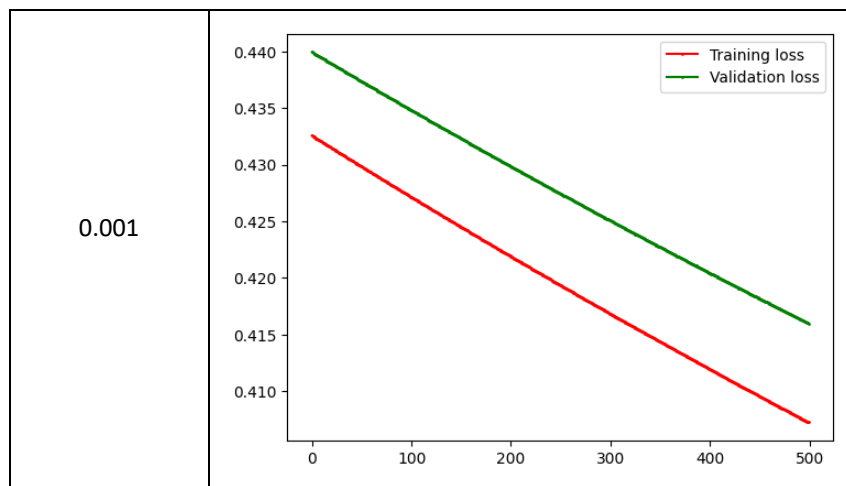
Datase : “logistic_data_B”



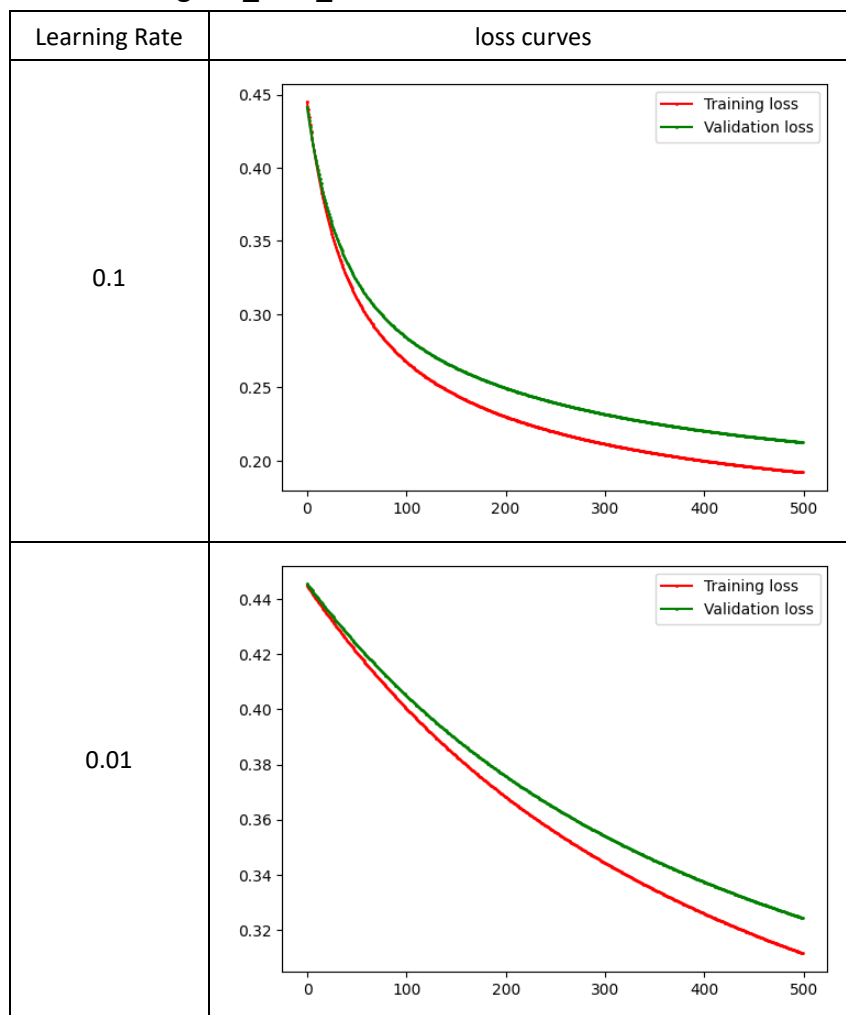


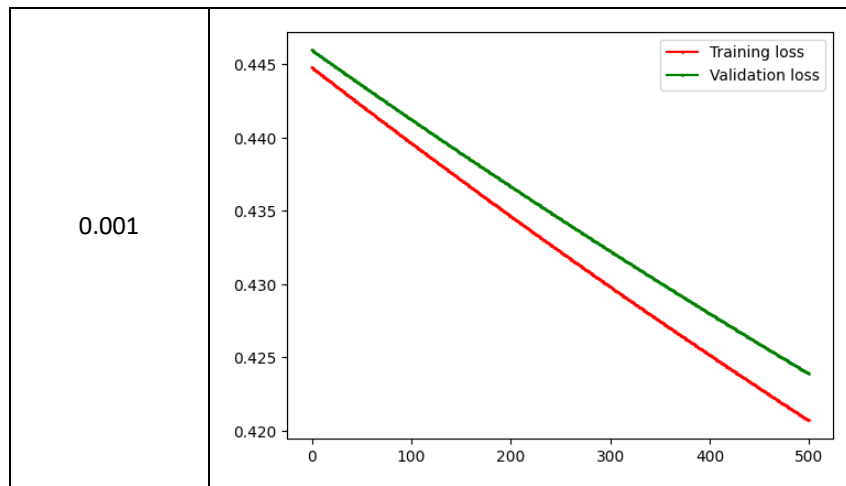
Datase : "logistic_data_C"





Datase : "logistic_data_D"





3. evaluation results in different learning rate

Datase : "logistic_data_A"

Learning rate	Results
0.1	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128
0.01	Logistic Regression Evaluation Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986 F1-score : 0.8884
0.001	Logistic Regression Evaluation Accuracy : 0.3875 Precision : 0.4364 Recall : 0.4424 F1-score : 0.4394

Datase : "logistic_data_B"

Learning rate	Results
0.1	Logistic Regression Evaluation Accuracy : 0.7800 Precision : 0.7793 Recall : 0.8160 F1-score : 0.7972

0.01	Logistic Regression Evaluation Accuracy : 0.7625 Precision : 0.7600 Recall : 0.8066 F1-score : 0.7826
0.001	Logistic Regression Evaluation Accuracy : 0.3925 Precision : 0.4286 Recall : 0.4387 F1-score : 0.4336

Datase : "logistic_data_C"

Learning rate	Results
0.1	Logistic Regression Evaluation Accuracy : 0.9762 Precision : 0.9852 Recall : 0.9719 F1-score : 0.9785
0.01	Logistic Regression Evaluation Accuracy : 0.9456 Precision : 0.9707 Recall : 0.9303 F1-score : 0.9500
0.001	Logistic Regression Evaluation Accuracy : 0.8406 Precision : 0.8774 Recall : 0.8290 F1-score : 0.8525

Datase : "logistic_data_D"

Learning rate	Results
0.1	Logistic Regression Evaluation Accuracy : 0.9225 Precision : 0.9330 Recall : 0.9266 F1-score : 0.9298

0.01	Logistic Regression Evaluation Accuracy : 0.9113 Precision : 0.9346 Recall : 0.9029 F1-score : 0.9185
0.001	Logistic Regression Evaluation Accuracy : 0.8319 Precision : 0.8668 Recall : 0.8228 F1-score : 0.8442

(A) Observations

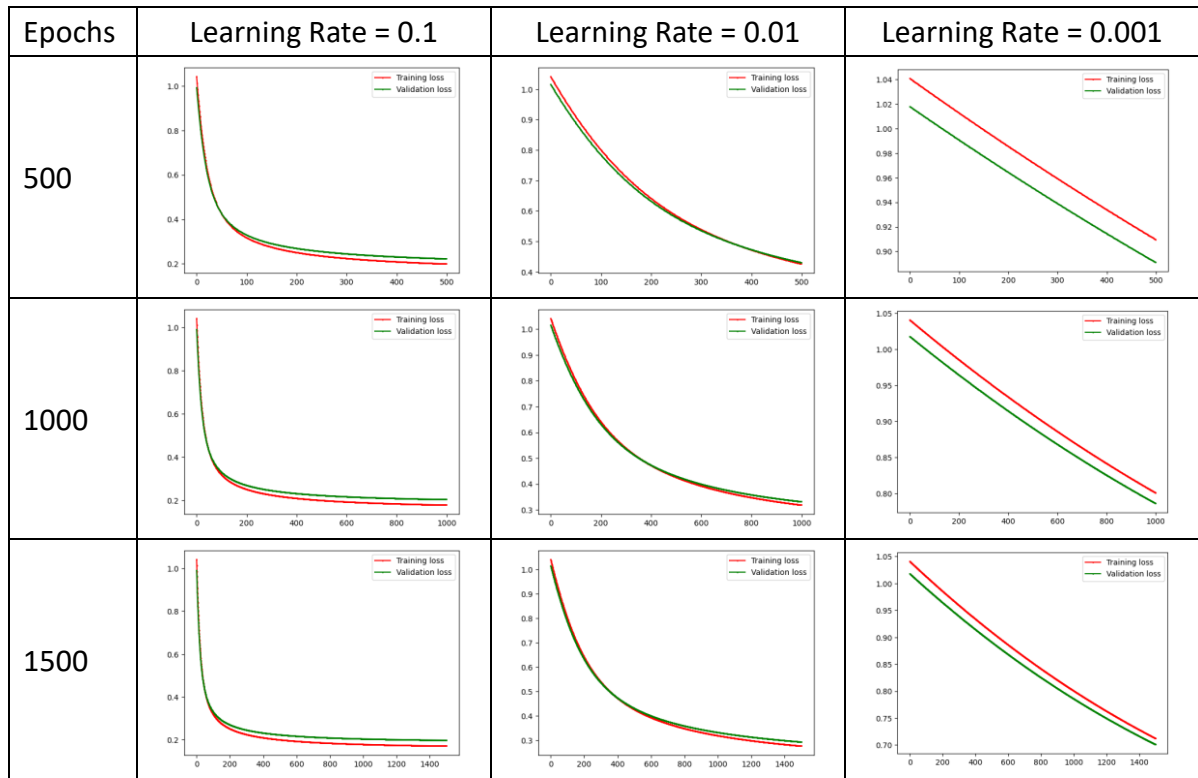
In my observation, **learning rate = 0.1** also gives the best overall balance between effectiveness and efficiency. In most cases, it produces the fastest loss reduction and also achieves the strongest or near-strongest evaluation scores, while **0.01** is still effective but converges more slowly, and **0.001** is generally the weakest because its loss decreases too slowly within the same training budget and its final scores are consistently lower. Among the four datasets, “**logistic_data_C**” performs the best overall, achieving the highest classification results at **lr = 0.1 (Accuracy = 0.9762, Precision = 0.9852, Recall = 0.9719, F1-score = 0.9785)**. This is consistent with its loss curves, where the model converges well and reaches a relatively low final loss. “**logistic_data_D**” is the next strongest dataset, also showing high scores and stable convergence, while “**logistic_data_A**” performs well but below C and D. In contrast, “**logistic_data_B**” is the weakest overall, with noticeably lower scores even under its best learning-rate setting, indicating that this dataset is more difficult for the model to separate effectively.

However, in terms of possible overfitting, the clearest suspicion appears in “**logistic_data_C**” and “**logistic_data_D**” under **lr = 0.1**. In both datasets, the training loss stays consistently below the validation loss with a visible gap throughout training, which suggests that the model fits the training data better than the validation data. However, because their final evaluation scores are still very high, this is better described as mild overfitting tendency rather than severe overfitting. By comparison, “**logistic_data_B**” shows much smaller train-validation gaps but also much lower overall performance, so its limitation is more likely underfitting or insufficient separability rather than overfitting. Overall, the most effective and efficient setting is “**logistic_data_C**” with learning rate = **0.1**, while the strongest signs of overfitting tendency are observed in **datasets C** and

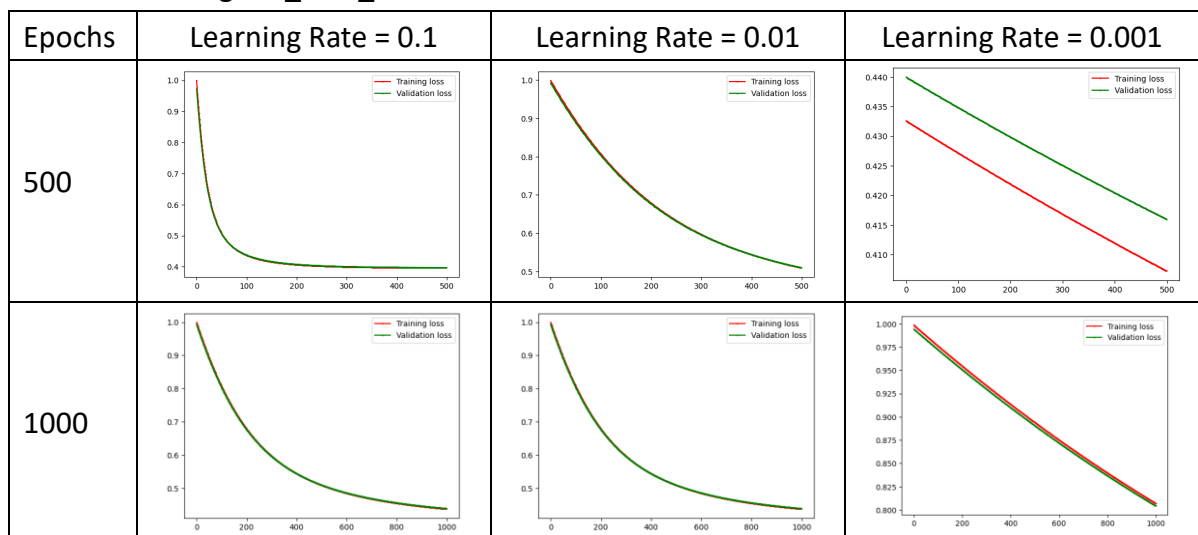
D, especially when the learning rate is high enough to let the model converge quickly.

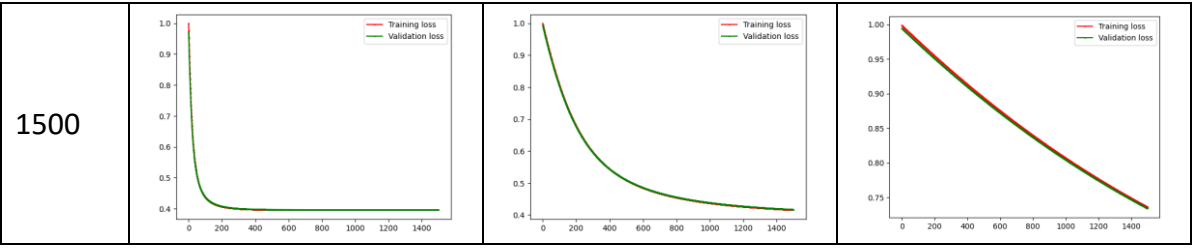
(B) number of training iteration in different learning rate

Datase : "logistic_data_A"

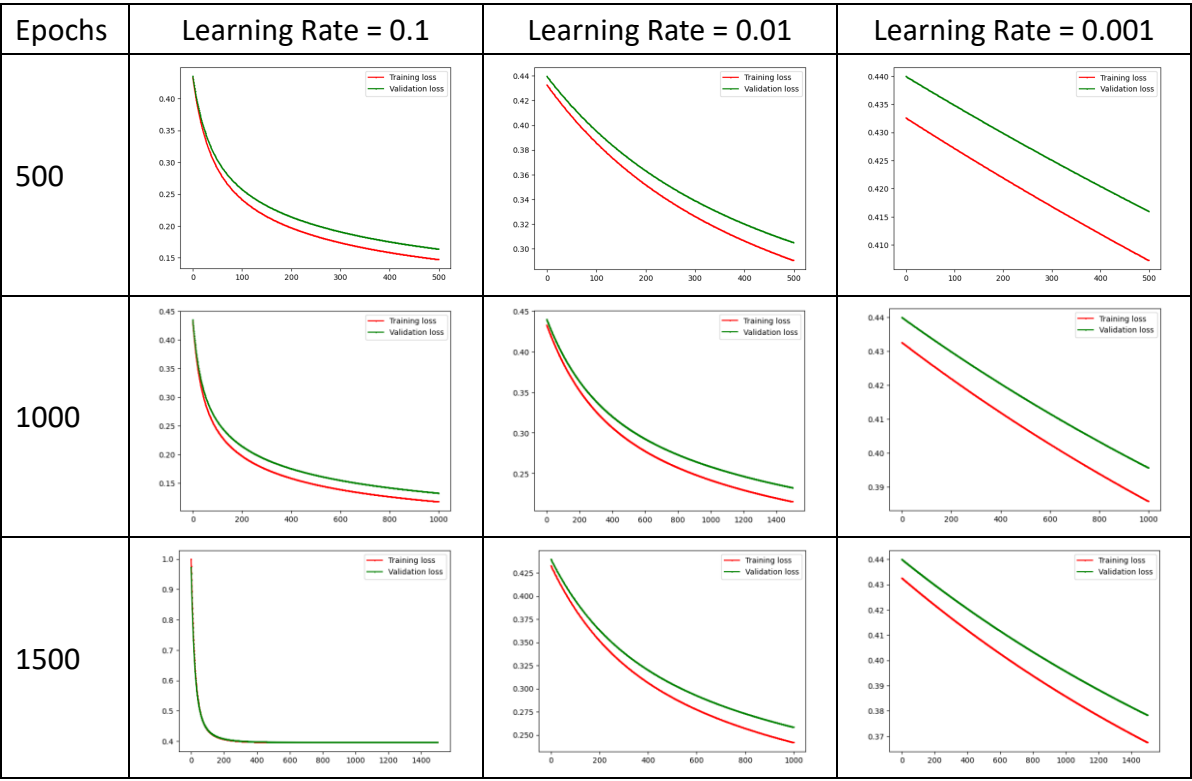


Datase : "logistic_data_B"

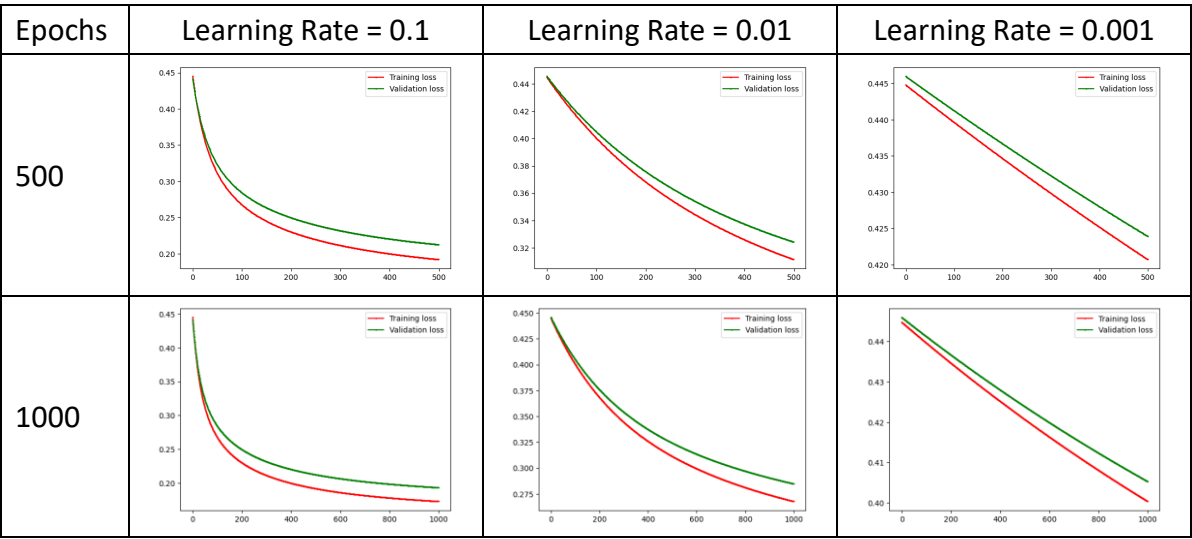


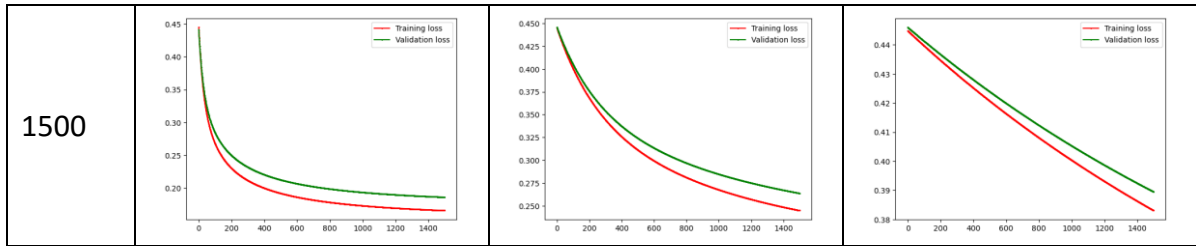


Datase : "logistic_data_C"



Datase : "logistic_data_D"





(C) evaluation the number of training iteration in different learning rate

Datase : "logistic_data_A"

Learning rate	Results epochs=500	Results epochs=1000	Results epochs=1500
0.1	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128	Logistic Regression Evaluation Accuracy : 0.7800 Precision : 0.7793 Recall : 0.8160 F1-score : 0.7972
0.01	Logistic Regression Evaluation Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986 F1-score : 0.8884	Logistic Regression Evaluation Accuracy : 0.9025 Precision : 0.9045 Recall : 0.9171 F1-score : 0.9108	Logistic Regression Evaluation Accuracy : 0.9025 Precision : 0.9083 Recall : 0.9124 F1-score : 0.9103
0.001	Logistic Regression Evaluation Accuracy : 0.3875 Precision : 0.4364 Recall : 0.4424 F1-score : 0.4394	Logistic Regression Evaluation Accuracy : 0.4625 Precision : 0.5044 Recall : 0.5253 F1-score : 0.5147	Logistic Regression Evaluation Accuracy : 0.5350 Precision : 0.5683 Recall : 0.5945 F1-score : 0.5811

Datase : "logistic_data_B"

Learning rate	Results epochs=500	Results epochs=1000	Results epochs=1500
0.1	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128	Logistic Regression Evaluation Accuracy : 0.7800 Precision : 0.7793 Recall : 0.8160 F1-score : 0.7972	Logistic Regression Evaluation Accuracy : 0.7800 Precision : 0.7793 Recall : 0.8160 F1-score : 0.7972
0.01	Logistic Regression Evaluation Accuracy : 0.7625 Precision : 0.7600 Recall : 0.8066 F1-score : 0.7826	Logistic Regression Evaluation Accuracy : 0.7775 Precision : 0.7783 Recall : 0.8113 F1-score : 0.7945	Logistic Regression Evaluation Accuracy : 0.7775 Precision : 0.7783 Recall : 0.8113 F1-score : 0.7945
0.001	Logistic Regression Evaluation Accuracy : 0.3925 Precision : 0.4286 Recall : 0.4387 F1-score : 0.4336	Logistic Regression Evaluation Accuracy : 0.4425 Precision : 0.4751 Recall : 0.4953 F1-score : 0.4850	Logistic Regression Evaluation Accuracy : 0.5075 Precision : 0.5330 Recall : 0.5708 F1-score : 0.5513

Datase : "logistic_data_C"

Learning rate	Results epochs=500	Results epochs=1000	Results epochs=1500
---------------	--------------------	---------------------	---------------------

0.1	Logistic Regression Evaluation Accuracy : 0.9762 Precision : 0.9852 Recall : 0.9719 F1-score : 0.9785	Logistic Regression Evaluation Accuracy : 0.9750 Precision : 0.9840 Recall : 0.9708 F1-score : 0.9773	Logistic Regression Evaluation Accuracy : 0.9756 Precision : 0.9841 Recall : 0.9719 F1-score : 0.9779
0.01	Logistic Regression Evaluation Accuracy : 0.9456 Precision : 0.9707 Recall : 0.9303 F1-score : 0.9500	Logistic Regression Evaluation Accuracy : 0.9744 Precision : 0.9885 Recall : 0.9651 F1-score : 0.9767	Logistic Regression Evaluation Accuracy : 0.9788 Precision : 0.9897 Recall : 0.9719 F1-score : 0.9807
0.001	Logistic Regression Evaluation Accuracy : 0.8406 Precision : 0.8774 Recall : 0.8290 F1-score : 0.8525	Logistic Regression Evaluation Accuracy : 0.8544 Precision : 0.8914 Recall : 0.8403 F1-score : 0.8651	Logistic Regression Evaluation Accuracy : 0.8756 Precision : 0.9059 Recall : 0.8661 F1-score : 0.8856

Datase : “logistic_data_D”

Learning rate	Results epochs=500	Results epochs=1000	Results epochs=1500
0.1	Logistic Regression Evaluation Accuracy : 0.9225 Precision : 0.9330 Recall : 0.9266 F1-score : 0.9298	Logistic Regression Evaluation Accuracy : 0.9219 Precision : 0.9319 Recall : 0.9266 F1-score : 0.9293	Logistic Regression Evaluation Accuracy : 0.9231 Precision : 0.9330 Recall : 0.9278 F1-score : 0.9304
0.01	Logistic Regression Evaluation Accuracy : 0.9113 Precision : 0.9346 Recall : 0.9029 F1-score : 0.9185	Logistic Regression Evaluation Accuracy : 0.9263 Precision : 0.9394 Recall : 0.9266 F1-score : 0.9330	Logistic Regression Evaluation Accuracy : 0.9256 Precision : 0.9393 Recall : 0.9255 F1-score : 0.9323
0.001	Logistic Regression Evaluation Accuracy : 0.8319 Precision : 0.8668 Recall : 0.8228 F1-score : 0.8442	Logistic Regression Evaluation Accuracy : 0.8438 Precision : 0.8795 Recall : 0.8318 F1-score : 0.8550	Logistic Regression Evaluation Accuracy : 0.8600 Precision : 0.8894 Recall : 0.8533 F1-score : 0.8710

(D) Observations

For most datasets, learning rate = 0.1 leads to the fastest decrease in training and validation loss, and this is usually matched by strong evaluation performance within only 500 epochs, indicating high training efficiency. However, the benefit of further increasing the number of epochs depends on the dataset. In “logistic_data_A” and “logistic_data_B”, the results suggest that extending training beyond the point of early convergence does not necessarily improve performance; for example, under **learning rate** = 0.1, both datasets show rapid convergence in the loss curves, yet the evaluation scores either remain unchanged or even decrease at later epochs, implying that additional training provides little benefit and may introduce a slight overfitting tendency or instability. In contrast, “logistic_data_C” and “logistic_data_D” show a more gradual and meaningful improvement when the learning rate is moderate, especially under **learning rate** = 0.01, where the losses continue to decline steadily as epochs increase and the evaluation metrics improve

accordingly. This pattern indicates that these datasets benefit from a longer optimization process and that their final scores are well aligned with the continued reduction in loss.

Overall, across all datasets, **learning rate = 0.001** is consistently the least effective choice: its loss curves decrease too slowly, and even when the number of epochs is increased from **500** to **1500**, the evaluation scores remain clearly below those obtained by **0.1** or **0.01**. This indicates insufficient convergence within the same training budget. Considering both effectiveness and efficiency, the strongest overall setting appears in “**logistic_data_C**”, whose best result is achieved with **learning rate = 0.01** at **1500** epochs (**Accuracy = 0.9788, Precision = 0.9897, Recall = 0.9719, F1-score = 0.9807**). This is consistent with its loss curves, where the model continues to improve as training proceeds and does not show abrupt degradation. By comparison, “**logistic_data_D**” also performs strongly and remains stable across several settings, while “**logistic_data_A**” and “**logistic_data_B**” reach their useful performance earlier and gain less from extended training. Therefore, the figures together show that the best training strategy depends on both learning rate and dataset characteristics: **learning rate = 0.1** is generally the most efficient, while **learning rate = 0.01** can produce the best final performance when more epochs are allowed, especially for datasets that continue to improve with longer training.

3. Data Preprocessing :

(A) median and Standard deviation for column with miss value

For the column that having miss value, they are “SepalLengthCm”, “SepalWidthCm”, “PetalLengthCm”, “PetalWidthCm” and “BranchLength”. For “SepalLengthCm” **median = 6.3000** and **std = 1.0371**. For “SepalWidthCm” **median = 2.9000** and **std = 0.3896**. For “PetalLengthCm” **median = 5.0856** and **std = 1.5828**. For “BranchLength” **median = 1.6000** and **std = 0.7067**. For “BranchLength” **median = 16.3000** and **std = 1.0352**.

```
Columns with missing values before imputation:
SepalLengthCm: median = 6.3000, std = 1.0371
SepalWidthCm: median = 2.9000, std = 0.3896
PetalLengthCm: median = 5.0856, std = 1.5828
PetalWidthCm: median = 1.6000, std = 0.7067
BranchLength: median = 16.3000, std = 1.0352
```

(B) replace the missing values using “Nearest Neighbors Imputation”

For the column after filling the missing value by **KNN(N = 5)**, “SepalLengthCm” **median = 6.3000** and **std = 1.0093**. For “SepalWidthCm” **median = 2.9000** and **std = 0.3724**. For “PetalLengthCm” **median = 5.0357** and **std = 1.5150**. For

“BranchLength” **median** = **1.7000** and **std** = **0.6946**. For “BranchLength” **median** = **16.3000** and **std** = **1.0110**.

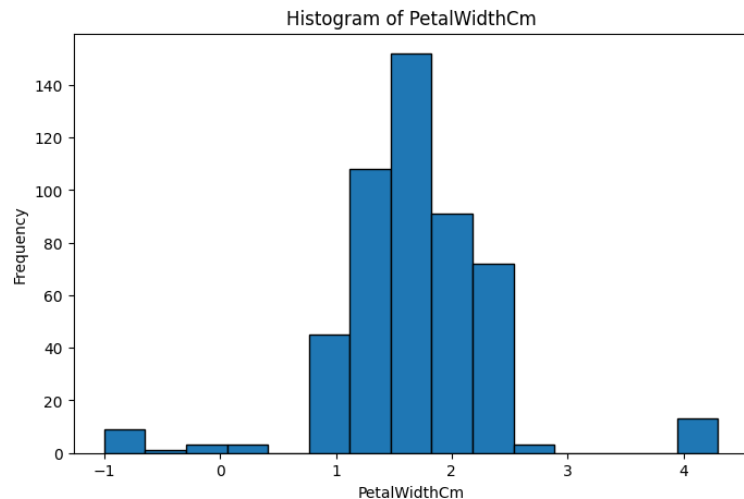
```
Columns with missing values after KNN imputation:  
SepallLengthCm: median = 6.3000, std = 1.0093  
SepalWidthCm: median = 2.9000, std = 0.3724  
PetalLengthCm: median = 5.0357, std = 1.5150  
PetalWidthCm: median = 1.7000, std = 0.6946  
BranchLength: median = 16.3000, std = 1.0110
```

In my observations, similar pattern can be observed in the **standard deviation**. After KNN imputation, the standard deviation of all affected columns becomes slightly smaller: **SepalLengthCm** decreases from **1.0371** to **1.0093**, **SepalWidthCm** from **0.3896** to **0.3724**, **PetalLengthCm** from **1.5828** to **1.5150**, **PetalWidthCm** from **0.7067** to **0.6946**, and **BranchLength** from **1.0352** to **1.0110**. This suggests that KNN imputation slightly reduces data variability, which is expected because missing values are replaced by estimates derived from neighboring observations, making the completed data somewhat smoother. Overall, the comparison shows that **KNN imputation preserves the general distribution of the dataset while modestly reducing dispersion**, indicating that it is an effective method for handling missing values without causing major statistical distortion.

2. Data Exploration :

(A) histogram of the column “PetalWidthCm” in “NYCU_Iris.csv”

The histogram shows that **PetalWidthCm** is mainly concentrated between approximately **1.0 and 2.5**, with the highest frequency between **1 to 2**. Indicating that most samples fall within a relatively narrow central range. The distribution is **not perfectly symmetric**, and it appears to be **slightly right-skewed**, as there are a few larger values extending toward the right side. In addition, the histogram contains a small number of values far from the main cluster, including some unusually low values near or below **0** and one very large value above **4**, which may be considered potential outliers or abnormal observations. Overall, the feature distribution suggests that most observations are grouped around the middle range, while a limited number of extreme values are present at both ends.



(B) report the largest positive correlation with the “PetalWidthCm” column

The largest Pearson correlation coefficient with “PetalWidthCm” that is 0.9917 from “PetalWidthCompactness”.

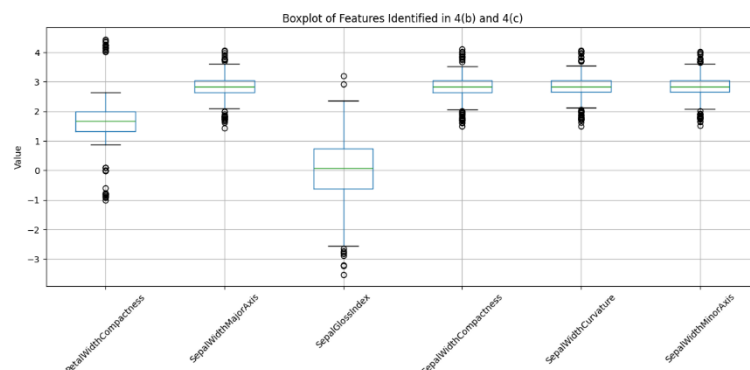
```
Feature with the largest positive correlation with PetalWidthCm:
PetalWidthCompactness: 0.9917
```

(C) top 5 features with the strongest negative correlations with the 'PetalWidthCm' column.

The top 5 features that having negative correlation with “PetalWidthCm” they are, **“SepalWidthMajorAxis” = -0.096350**, **“SepalGlossIndex” = -0.095200**, **“SepalWidthCompactness” = -0.088496**, **“SepalWidthCurvature” = -0.081334** and **“SepalWidthMinorAxis” = -0.074392**.

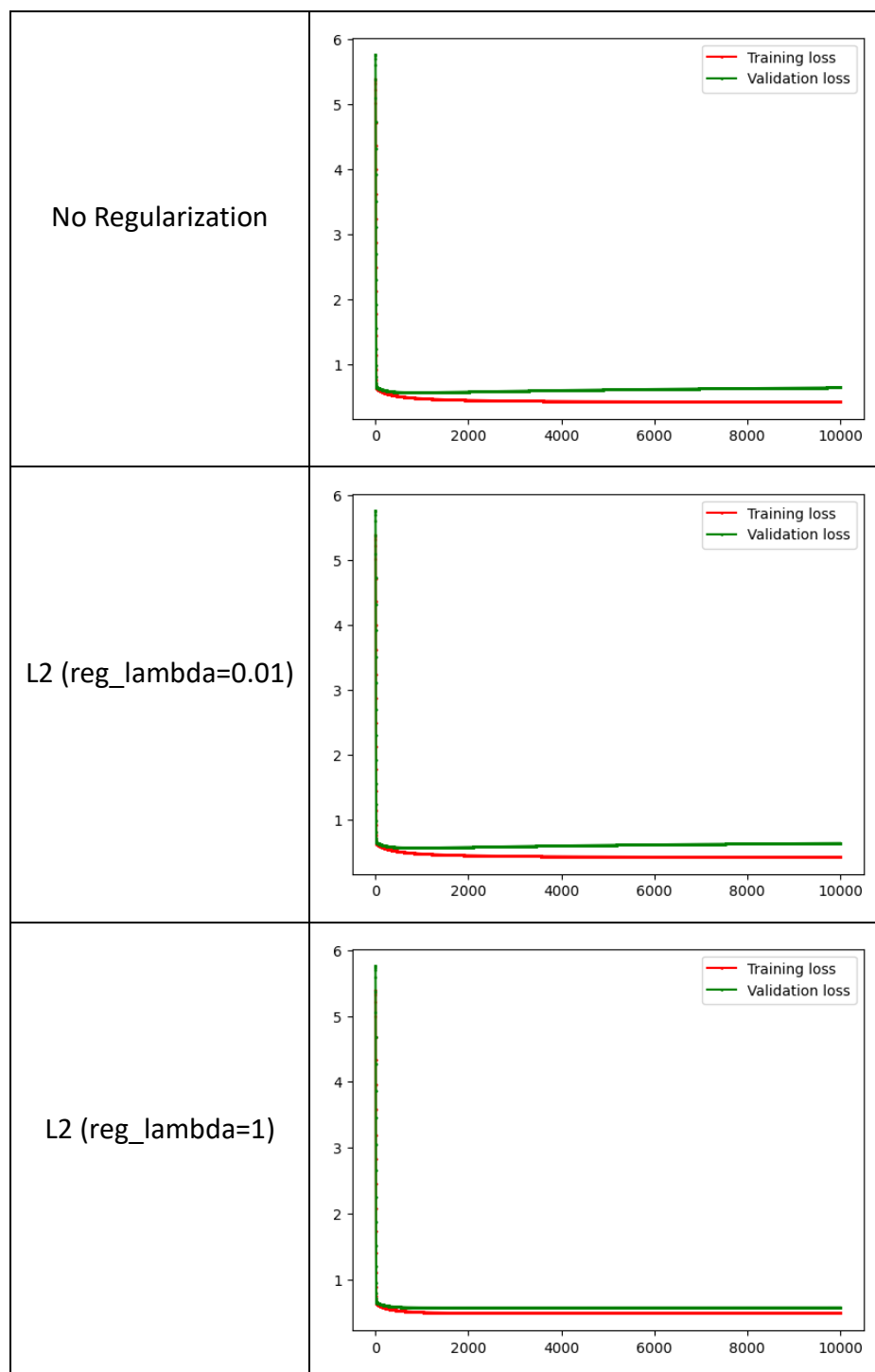
```
Top 5 features with the strongest negative correlations with PetalWidthCm:
SepalWidthMajorAxis    -0.096350
SepalGlossIndex        -0.095200
SepalWidthCompactness  -0.088496
SepalWidthCurvature    -0.081334
SepalWidthMinorAxis    -0.074392
```

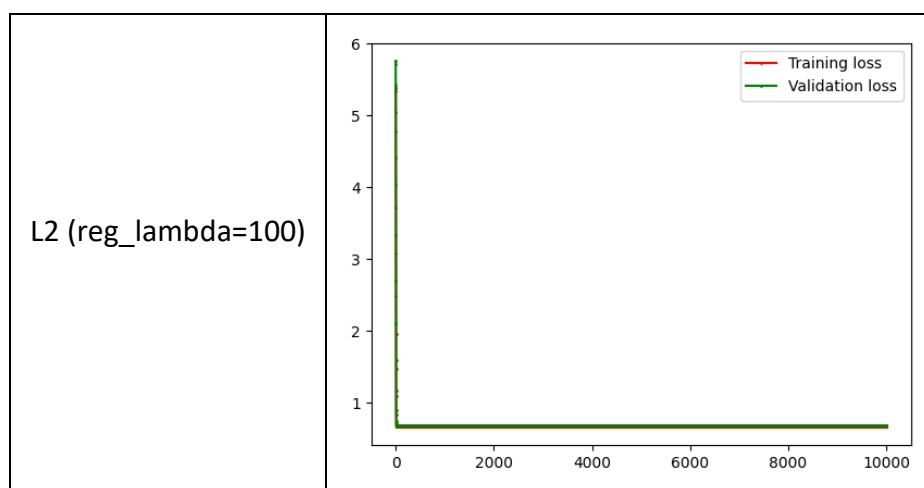
(D) boxplot for all features found in 4(b) and 4(c)



3. Regularization :

(A) L2 Regularization





(B) evaluation the number of training iteration in different learning rate

No Regularization	L2(reg_lambda=0.01)	L2 (reg_lambda=1)	L2(reg_lambda=100)
Model Evaluation Accuracy : 0.7200 Precision : 0.7125 Recall : 0.7500 F1-score : 0.7308	Model Evaluation Accuracy : 0.7200 Precision : 0.7125 Recall : 0.7500 F1-score : 0.7308	Model Evaluation Accuracy : 0.7400 Precision : 0.7342 Recall : 0.7632 F1-score : 0.7484	Model Evaluation Accuracy : 0.5067 Precision : 0.5067 Recall : 1.0000 F1-score : 0.6726

(C) Observations

In Table (5A), all settings show a rapid decrease in both training and validation loss at the beginning of training, indicating that the model can fit the data efficiently regardless of the regularization strength. However, the relationship between the training and validation curves differs across settings, and this difference is reflected in the final metrics reported in Table (5B). With **no regularization** and **L2 regularization with $\lambda = 0.01$** , the loss curves are almost identical, and the final evaluation results are exactly the same (**Accuracy = 0.7200, Precision = 0.7125, Recall = 0.7500, F1-score = 0.7308**). This indicates that such a small regularization strength has little practical effect on the model behavior or predictive performance. When the regularization strength is increased to **$\lambda = 1$** , the training and validation losses remain close and stable while the final classification metrics improve to **Accuracy = 0.7400, Precision = 0.7342, Recall = 0.7632, and F1-score = 0.7484**, suggesting that moderate L2 regularization improves generalization by constraining the model parameters without excessively limiting learning capacity. In contrast, when the regularization becomes too strong (**$\lambda = 100$**), the training and validation losses quickly flatten at a relatively higher level and almost overlap completely, indicating that the model is overly constrained and unable to learn sufficiently rich decision boundaries. This behavior is directly confirmed by Table (5B), where the performance drops

substantially to **Accuracy = 0.5067** and **F1-score = 0.6726**. Although the recall reaches **1.0000**, the precision falls to **0.5067**, implying that the model predicts the positive class too aggressively and sacrifices overall discriminative ability. Overall, Table (5A) explains how different levels of L2 regularization affect the optimization dynamics, while Table (5B) confirms their impact on final performance. Taken together, the results show that **moderate regularization ($\lambda = 1$)** provides the best balance between fitting ability and generalization, whereas **too little regularization** offers no clear benefit and **too much regularization** leads to underfitting.